



BITS Pilani
Pilani Campus

Support Vector Machines

MFML Team

Agenda



- ▶ Last class we set the SVM Primal problem
- ▶ This class we turn our attention to how to solve this problem
- ▶ Also we will study what is soft SVM and how to solve this problem

Solving the Optimization Problem



Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$ is minimized; Type equation here.

and for all $\{(\mathbf{x}_i, y_i)\}$: $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

← Primal

- Need to optimize a *quadratic* function subject to *linear inequality* constraints.
- All constraints in SVM are linear
- Quadratic optimization problems are a well-known class of mathematical programming problems, and many (rather intricate) algorithms exist for solving them.
- The solution involves constructing a *unconstrained problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:

Recall : Karush–Kuhn–Tucker (KKT) theorem

- $\max f(x)$ subject to $g_i(x) = 0$ and $h_j(x) \geq 0$ for all i, j
- Make the Lagrangian function

$$\mathcal{L} = f(x) - \sum_i \lambda_i g_i(x) - \sum_j \mu_j h_j(x)$$

- Necessary conditions to have a minimum are

$$\nabla_x \mathcal{L}(x^*, \lambda^*, \mu^*) = 0$$

$$g_i(x^*) = 0 \text{ for all } i$$

$$h_j(x^*) \geq 0 \text{ for all } j$$

$$\mu_j \geq 0 \text{ for all } j$$

$$\mu_j^* h_j(x^*) = 0 \text{ for all } j$$

Solving the Optimization Problem

- The solution involves constructing a *dual problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1]$$

- Taking partial derivative with respect to $\mathbf{w}$, $\frac{\partial L}{\partial \mathbf{w}} = 0$
 - $\mathbf{w} - \sum \alpha_i y_i \mathbf{x}_i = 0$
 - $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$
- Taking partial derivative with respect to b , $\frac{\partial L}{\partial b} = 0$
 - $-\sum \alpha_i y_i = 0$
 - $\sum \alpha_i y_i = 0$

Solving the Optimization Problem

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum \alpha_i [y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1]$$

- Expanding above equation:

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum \alpha_i y_i \mathbf{w} \cdot \mathbf{x}_i - \sum \alpha_i y_i b + \sum \alpha_i$$

- Substituting $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$ and $\sum \alpha_i y_i = 0$ in above equation

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2} \left(\sum_i \alpha_i y_i \mathbf{x}_i \right) \left(\sum_j \alpha_j y_j \mathbf{x}_j \right) - \left(\sum_i \alpha_i y_i \mathbf{x}_i \right) \left(\sum_j \alpha_j y_j \mathbf{x}_j \right) + \sum \alpha_i$$

$$L(\mathbf{w}, b, \alpha_i) = \sum \alpha_i - \frac{1}{2} \left(\sum_i \alpha_i y_i \mathbf{x}_i \right) \left(\sum_j \alpha_j y_j \mathbf{x}_j \right)$$

$$L(\mathbf{w}, b, \alpha_i) = \sum \alpha_i - \frac{1}{2} \left(\sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \right)$$

Support Vectors



Using KKT conditions : $\alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0$

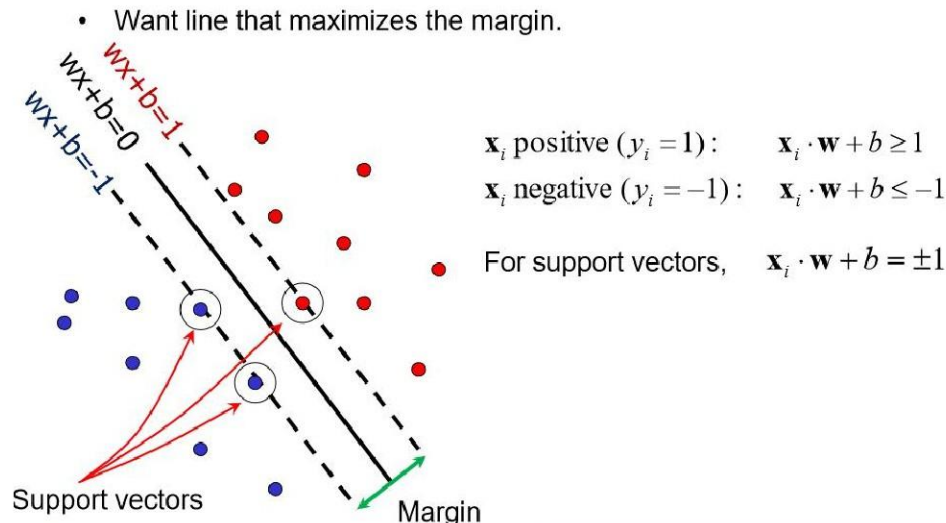
For this condition to be satisfied either $\alpha_i = 0$ and $y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 > 0$

OR $y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$ and $\alpha_i > 0$

For support vectors: $y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1]$$

For all points other than support vectors: $\alpha_i = 0$



Solving the Optimization Problem

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$

Learned
weight

Support
vector

Solving the Optimization Problem

- Solution: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$
 $b = y_i - \mathbf{w} \cdot \mathbf{x}_i$ (for any support vector)

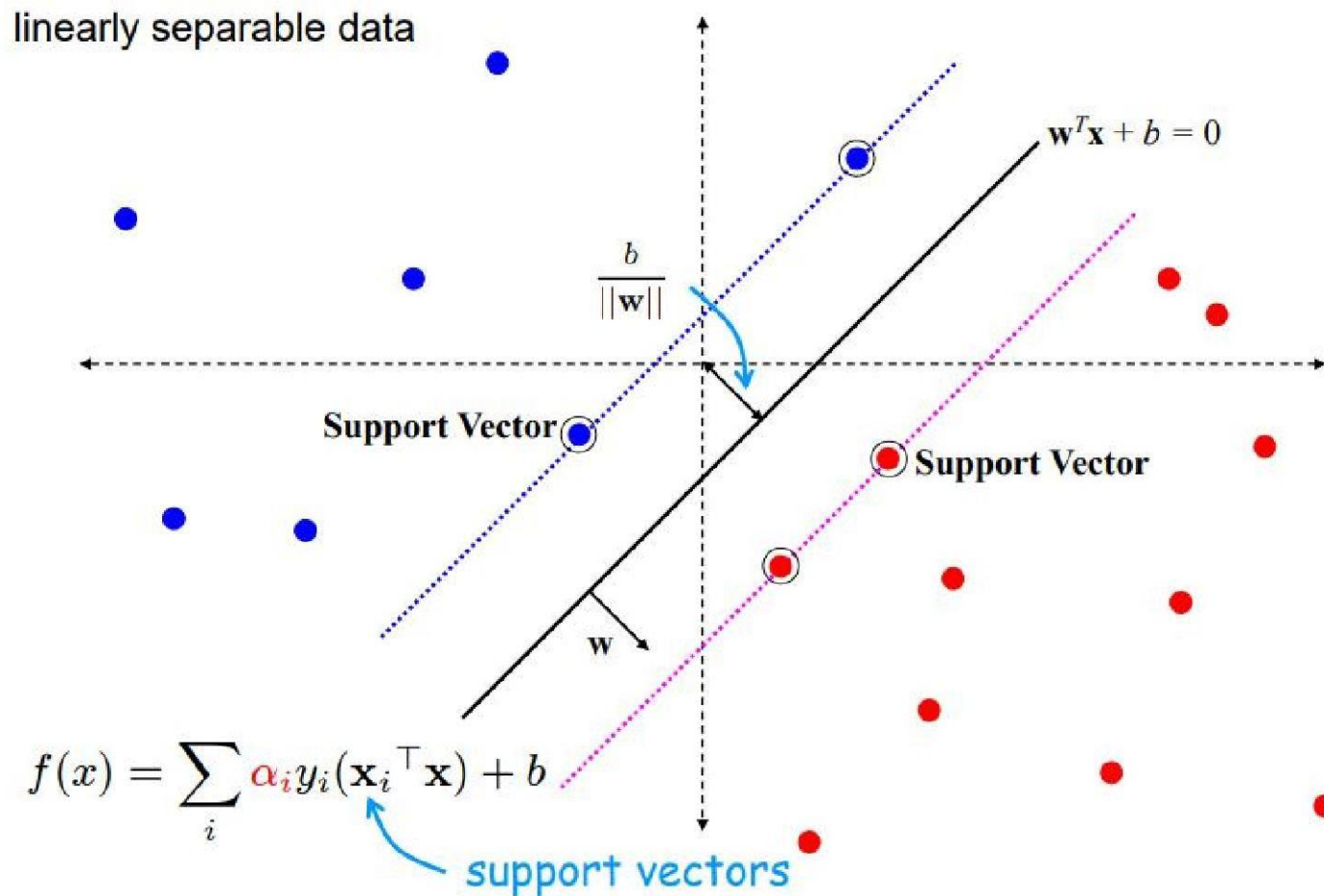
- Classification function:

$$\begin{aligned} f(x) &= \text{sign}(\mathbf{w} \cdot \mathbf{x} + b) \\ &= \text{sign}\left(\sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b\right) \end{aligned}$$

If $f(x) < 0$, classify as negative, otherwise classify as positive.

- Notice that it relies on an *inner product* between the test point $\mathbf{x}$ and the support vectors $\mathbf{x}_i$
- (Solving the optimization problem also involves computing the inner products $\mathbf{x}_i \cdot \mathbf{x}_j$ between all pairs of training points)

Substituting w in support vectors function



Linear SVM: Numerical Problem



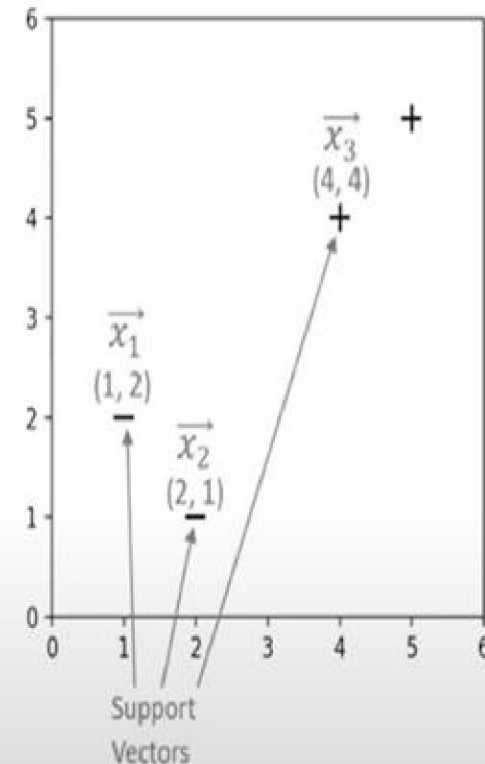
$$\sum_i \alpha_i y_i \vec{x}_i \cdot \vec{x} + b = 1 \quad \text{if } \vec{x} \text{ is +ve support vector}$$

$$\sum_i \alpha_i y_i \vec{x}_i \cdot \vec{x} + b = -1 \quad \text{if } \vec{x} \text{ is -ve support vector}$$

$$\sum_i \alpha_i y_i = 0$$

$$\text{Let } \beta_i = \alpha_i y_i$$

$$\begin{cases} \beta_1 \vec{x}_1 \cdot \vec{x}_1 + \beta_2 \vec{x}_2 \cdot \vec{x}_1 + \beta_3 \vec{x}_3 \cdot \vec{x}_1 + b = -1 & \vec{x} = \vec{x}_1 \\ \beta_1 \vec{x}_1 \cdot \vec{x}_2 + \beta_2 \vec{x}_2 \cdot \vec{x}_2 + \beta_3 \vec{x}_3 \cdot \vec{x}_2 + b = -1 & \vec{x} = \vec{x}_2 \\ \beta_1 \vec{x}_1 \cdot \vec{x}_3 + \beta_2 \vec{x}_2 \cdot \vec{x}_3 + \beta_3 \vec{x}_3 \cdot \vec{x}_3 + b = 1 & \vec{x} = \vec{x}_3 \\ \beta_1 + \beta_2 + \beta_3 = 0 \end{cases}$$



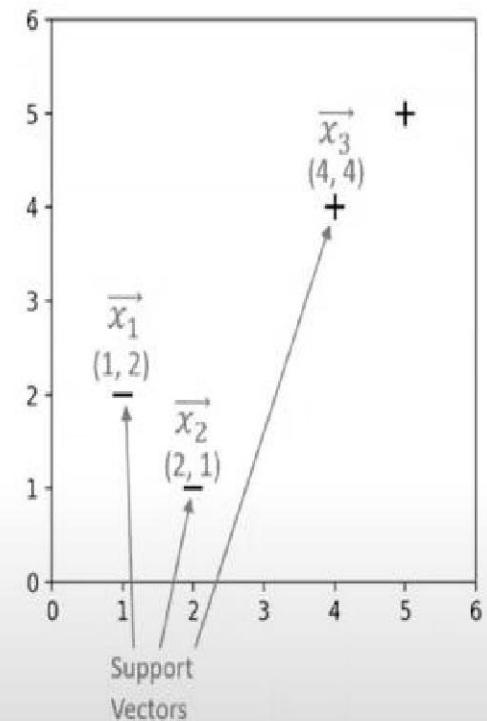
Numerical: Linear SVM



$$\begin{cases} \beta_1 \vec{x}_1 \cdot \vec{x}_1 + \beta_2 \vec{x}_2 \cdot \vec{x}_1 + \beta_3 \vec{x}_3 \cdot \vec{x}_1 + b = -1 \\ \beta_1 \vec{x}_1 \cdot \vec{x}_2 + \beta_2 \vec{x}_2 \cdot \vec{x}_2 + \beta_3 \vec{x}_3 \cdot \vec{x}_2 + b = -1 \\ \beta_1 \vec{x}_1 \cdot \vec{x}_3 + \beta_2 \vec{x}_2 \cdot \vec{x}_3 + \beta_3 \vec{x}_3 \cdot \vec{x}_3 + b = 1 \\ \beta_1 + \beta_2 + \beta_3 = 0 \end{cases}$$

Plug in the values of all the vectors

$$\begin{cases} 5\beta_1 + 4\beta_2 + 12\beta_3 + b = -1 \\ 4\beta_1 + 5\beta_2 + 12\beta_3 + b = -1 \\ 12\beta_1 + 12\beta_2 + 32\beta_3 + b = 1 \\ \beta_1 + \beta_2 + \beta_3 = 0 \end{cases} \Rightarrow \begin{cases} \beta_1 = -0.08 \\ \beta_2 = -0.08 \\ \beta_3 = 0.16 \\ b = -2.2 \end{cases}$$



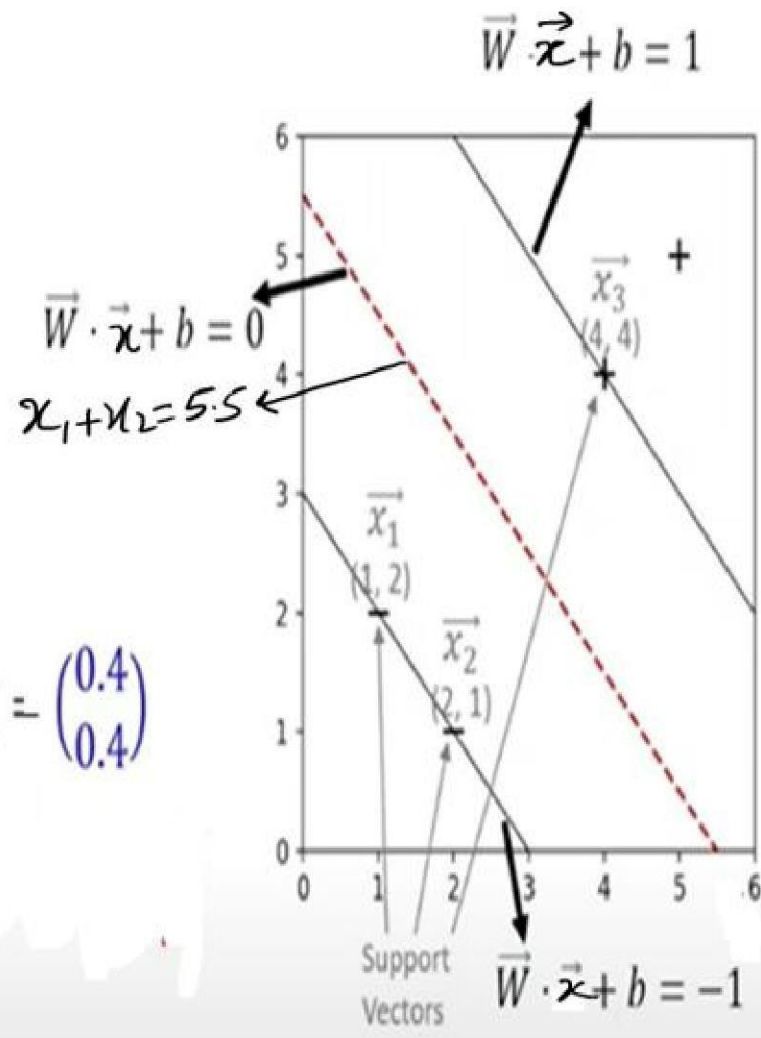
$$\begin{cases} \beta_1 = -0.08 \\ \beta_2 = -0.08 \\ \beta_3 = 0.16 \end{cases}$$

So, hyperplane is essentially those $\vec{x}$ that satisfy the equality.

$$\vec{W} = \sum_i \alpha_i y_i \vec{x}_i \quad \sum_i \beta_i \vec{x}_i = -0.08 \begin{pmatrix} 1 \\ 2 \end{pmatrix} - 0.08 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 0.16 \begin{pmatrix} 4 \\ 4 \end{pmatrix} = \begin{pmatrix} 0.4 \\ 0.4 \end{pmatrix}$$

$$: \vec{W} \cdot \vec{x} + b = 0$$

So, $\begin{pmatrix} 0.4 \\ 0.4 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - 2.2 = 0 \implies x_1 + x_2 = 5.5$



Hinge Loss



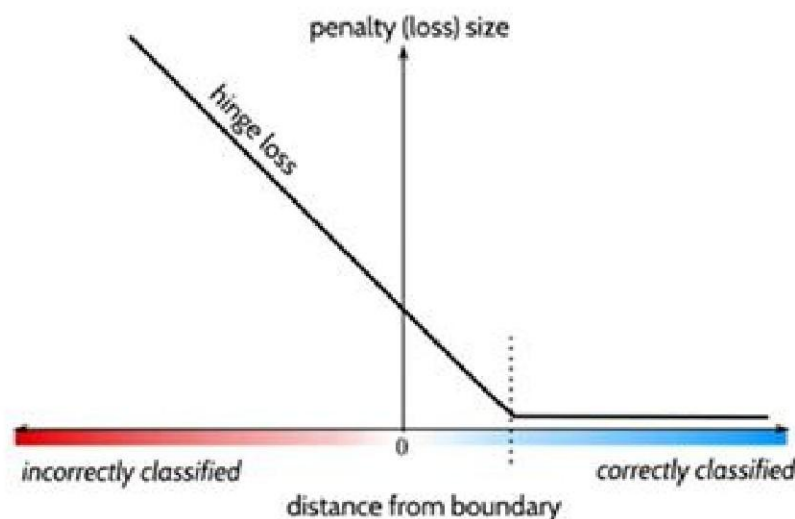
Hinge Loss is one of the types of Loss Function, mainly used for **maximum margin** classification models.

Hinge Loss incorporates a margin or distance from the classification boundary into the loss calculation. Even if new observations are classified correctly, they can incur a penalty if the margin from the decision boundary is not large enough.

$$L = \max(0, 1 - y_i (w^T x_i + b))$$

0 - for correct classification

1 - for wrong classification



Hinge loss

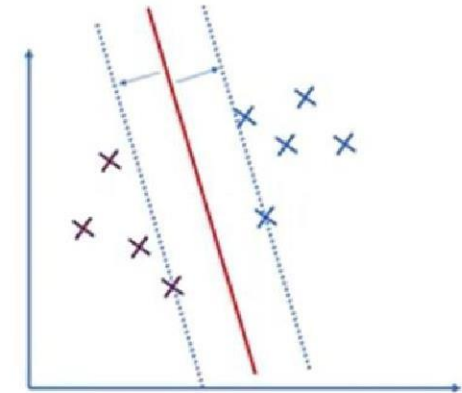


HINGE LOSS – Numerical Example 1

$$y' = W^T x + b$$

Correctly classification

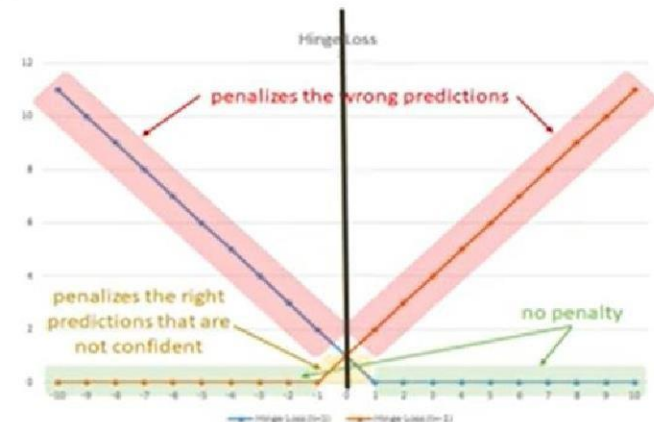
- Actual output $y = +1$ Predicted output $y' = 0.5$. ✓
- Hinge Loss = $\max(0, 1 - y * y')$
 $= \max(0, 1 - 1 * 0.5)$
 $= 0.5$.



Hinge loss is **close to zero** for correctly classified sample.

Misclassification

- Actual output $y = -1$. Predicted output $y' = 0.5$.
- Hinge Loss = $\max(0, 1 - y * y')$
 $= \max(0, 1 - (-1) * 0.5)$
 $= 1.5$.
- Hinge loss is **higher** for misclassified sample.



SVM : Estimating w and b



n: no. of data points

- Want to minimize $J(w, b)$:

$$J(w, b) = \frac{1}{2} \sum_{j=1}^d (w^{(j)})^2 + C \sum_{i=1}^n \max \left\{ 0, 1 - y_i \left(\sum_{j=1}^d w^{(j)} x_i^{(j)} + b \right) \right\}$$

Empirical loss $L(x_i, y_i)$

- Compute the gradient $\nabla(j)$ w.r.t. $w^{(j)}$

$$\frac{\partial J(w, b)}{\partial w^{(j)}} = w^{(j)} + C \sum_{i=1}^n \frac{\partial L(x_i, y_i)}{\partial w^{(j)}}$$

$$\frac{\partial J}{\partial b} = C \sum_{i=1}^n \frac{\partial L(x_i, y_i)}{\partial b}$$

$$\frac{\partial L(x_i, y_i)}{\partial b} = 0 \quad \text{if } y_i(w \cdot x_i + b) \geq 1$$

$$= -y_i \quad \text{otherwise}$$

$$\frac{\partial L(x_i, y_i)}{\partial w^{(j)}} = 0 \quad \text{if } y_i(w \cdot x_i + b) \geq 1$$

$$= -y_i x_i^{(j)} \quad \text{else}$$

Gradient Descent

Iterate until Convergence

$$\text{Compute : } \frac{\partial J}{\partial b} = C \sum_1^n \frac{\partial L(x_i, y_i)}{\partial b}$$

$$\text{Update : } b_{\text{new}} \leftarrow b - \eta \frac{\partial J}{\partial b},$$

For $j=1, \dots, d$

$$\text{Compute : } \frac{\partial J}{\partial w^j} = w^j + C \sum_1^n \frac{\partial L(x_i, y_i)}{\partial w^j}$$

$$\text{Update : } w_{\text{new}}^j \leftarrow w^j - \eta \frac{\partial J}{\partial w^j},$$

$$b \leftarrow b_{\text{new}}, w \leftarrow w_{\text{new}}$$

C parameter in cost function



C parameter tells the SVM optimization how much you want to avoid misclassifying each training example.

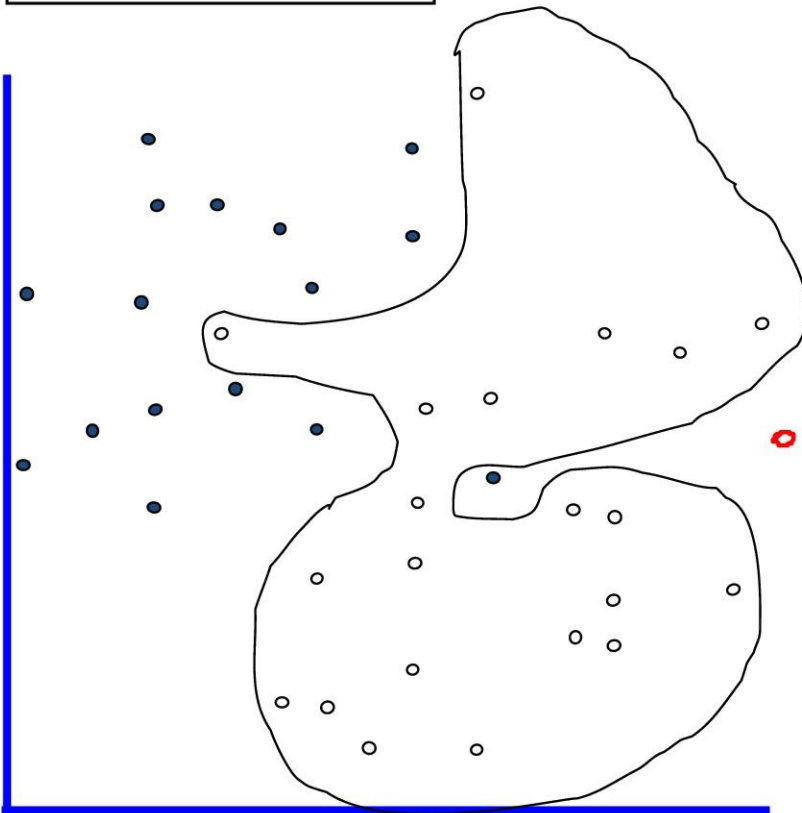
For large values of C , the optimization will choose a smaller-margin hyperplane if that hyperplane does a better job of getting all the training points classified correctly.

Conversely, a very small value of C will cause the optimizer to look for a larger-margin separating hyperplane, even if that hyperplane misclassifies more points

Dataset with noise



- denotes +1
- denotes -1



- **Hard Margin:** So far we require all data points be classified correctly
 - No training error
- **What if the training set is noisy?**

Soft Margin Classification

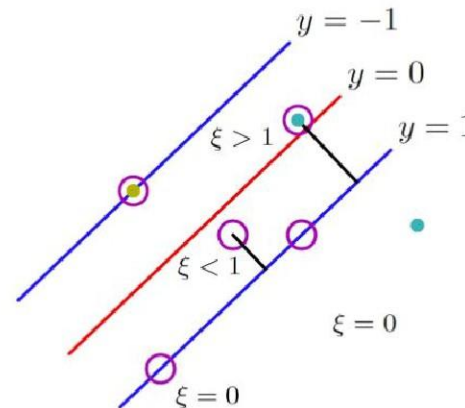


Slack variables ξ_i can be added to allow misclassification of difficult or noisy examples.

What should our quadratic optimization criterion be?

Minimize

$$\frac{1}{2} \mathbf{w} \cdot \mathbf{w} + C \sum_{k=1}^R \varepsilon_k$$



- **Slack variable** as giving the classifier some leniency when it comes to moving around points near the **margin**.
- When C is large, larger slacks penalize the objective function of SVM's more than when C is small.

Soft Margin



The w that minimizes...

$$\min_w \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{Maximize margin}} + \underbrace{C \sum_{i=1}^N \xi_i}_{\text{Minimize misclassification}}$$

Annotations:

- N : # data samples
- C : Misclassification cost
- ξ_i : Slack variable

subject to

$$y_i \mathbf{w}^T \mathbf{x}_i \geq 1 - \xi_i,$$
$$\xi_i \geq 0, \quad \forall i = 1, \dots, N$$

Hard Margin versus Soft Margin



- **Hard Margin:**

Find $\mathbf{w}$ and b such that

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \text{ is minimized and for all } \{(\mathbf{x}_i, y_i)\}$$
$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

- **Soft Margin incorporating slack variables:**

Find $\mathbf{w}$ and b such that

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum \xi_i \text{ is minimized and for all } \{(\mathbf{x}_i, y_i)\}$$
$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \text{ and } \xi_i \geq 0 \text{ for all } i$$

- **Parameter C can be viewed as a way to control overfitting.**

Given a set of 2D points $\{(1, 2), (2, 3), (3, 1), (4, 4)\}$ formulate the optimization problem for Support Vector Machine (SVM) algorithm with a soft margin, where the regularization parameter $C = 0.5$ and the kernel is linear. Assume that the points $(1, 2)$ and $(3, 1)$ belong to one class, and the points $(2, 3)$ and $(4, 4)$ belong to another class.

First we formulate the optimization problem.

To calculate the optimal hyperplane using the Support Vector Machine (SVM) algorithm, we first need to understand that the goal is to find the weights (w) and bias (b) that maximize the margin between the two classes, given the regularization parameter $C = 0.5$ and a linear kernel.

The points (1, 2) and (3, 1) belong to one class, which we can denote as class A with label $y = -1$, and the points (2, 3) and (4, 4) belong to another class, denoted as class B with label $y = 1$.

Formulation

Given the linear kernel, the decision boundary (hyperplane) can be represented as $w^T x + b = 0$, where w is the weight vector, x is the input vector, and b is the bias. For a 2D space, $w = [w_1, w_2]$, $x = [x_1, x_2]$, and the equation becomes $w_1 x_1 + w_2 x_2 + b = 0$.

Solution

We need to solve the optimization problem that maximizes the margin, which is formulated as:

Minimize: $(1/2) * w^T * w + C * \sum \xi_i$

Subject to:

$$y_i (w^T * x_i + b) \geq 1 - \xi_i$$

where ξ_i are slack variables introduced for the soft margin, and the sum is over all training examples.

Calculation

Let's assign the labels and calculate:

For (1, 2), $y = -1$

For (2, 3), $y = 1$

For (3, 1), $y = -1$

For (4, 4), $y = 1$

Using a linear kernel and considering the soft margin, we aim to find w and b that satisfy the constraints with minimal violation (ξ_i).

:

Given points: $x_1 = (1, 2)$, $x_2 = (2, 3)$, $x_3 = (3, 1)$, $x_4 = (4, 4)$

Class labels: $y_1 = 1$, $y_2 = -1$, $y_3 = 1$, $y_4 = -1$

The optimization problem becomes:

Minimize: $\frac{1}{2}(w_1^2 + w_2^2) + 0.5(\xi_1 + \xi_2 + \xi_3 + \xi_4)$

Subject to:

$$1(1w_1 + 2w_2 + b) \geq 1 - \xi_1$$

$$-1(2w_1 + 3w_2 + b) \geq 1 - \xi_2$$

$$1(3w_1 + 1w_2 + b) \geq 1 - \xi_3$$

$$-1(4w_1 + 4w_2 + b) \geq 1 - \xi_4$$

Solving this problem is little tedious.

You can run python code to solve this problem.

For your reference the python code and the ouputs are given in the following slides.

```
1 import numpy as np
2 from sklearn import svm
3
4 # Define the points
5 X = np.array([[1, 2], [2, 3], [3, 1], [4, 4]])
6
7 # Define the labels
8 y = np.array([-1, 1, -1, 1])
9
10 # Create an SVM classifier with a linear kernel and C = 0.5
11 clf = svm.SVC(kernel='linear', C=0.5)
12
13 # Fit the classifier to the data
14 clf.fit(X, y)
15
16 # Get the coefficients (weights) and the intercept (bias)
17 w = clf.coef_[0]
18 b = clf.intercept_[0]
19
20 print("Weights (w):", w)
21 print("Bias (b):", b)
```

Weights (w): [0.3 0.6]
Bias (b): -2.5